

```

    // bit of that value is 1, then the value is negative.

    // Unless it already fits, of course.
    println!(" 128 as a i16 is: {}", 128 as i16);

    // 128 as i8 -> -128, whose two's complement in eight
bits is:
    println!(" 128 as a i8 is : {}", 128 as i8);

    // repeating the example above
    // 1000 as u8 -> 232
    println!("1000 as a u8 is : {}", 1000 as u8);
    // and the two's complement of 232 is -24
    println!(" 232 as a i8 is : {}", 232 as i8);

    // Since Rust 1.45, the `as` keyword performs a *saturating cast*
    // when casting from float to int. If the floating point
value exceeds
    // the upper bound or is less than the lower bound, the
returned value
    // will be equal to the bound crossed.

    // 300.0 is 255
    println!("300.0 is {}", 300.0_f32 as u8);
    // -100.0 as u8 is 0
    println!("-100.0 as u8 is {}", -100.0_f32 as u8);
    // nan as u8 is 0
    println!("nan as u8 is {}", f32::NAN as u8);

    // This behavior incurs a small runtime cost and can
be avoided
    // with unsafe methods, however the results might over-
flow and
    // return **unsound values**. Use these methods wisely:
    unsafe {
        // 300.0 is 44
        println!("300.0 is {}", 300.0_f32.to_int_unchecked::());
        // -100.0 as u8 is 156
        println!("-100.0 as u8 is {}", (-100.0_f32).to_int_unchecked::());
    }

```